Cucumber als primäres Testtool

(Anm. ehemals Schulungsunterlage, dient nun als Handbuch und umfängliche Funktionsbeschreibung)

Inhalt

- Einleitung
  - Mail von `@Jürgen Nöding` vom 04.12.18
  - Handhabung für JUnit-Tests
- Funktionsweise von Cucumber
- Basis-Steps Cucumber
  - Code Completion
    - Erstes einfaches Testskript
  - Öffnen eines Editors
  - Auf erwartete Feldwerte testen
    - Feldwerte prüfen
  - Erfolgreiche Testausführung
    - Hinweis
    - Hinweis
  - Auf erwartete Fehler testen
    - Hinweis
    - Fehler beim Setzen eines Feldwerts
      - Erwartete Fehler bei der Feldeingabe abfangen
    - Fehler beim Speichern des Editors
      - Fehler beim Speichern des Editors abfangen
    - Fehler beim Öffnen des Editors mit einem bestimmten Kommando
      - Fehler beim Speichern des Editors abfangen
    - Fehler beim Einfügen oder Löschen von Tabellenzeilen
      - Abfangen von erwarteten Fehlern beim Erzeugen und Löschen von Tabellenzeilen
    - Fehler beim Klick auf Buttons
      - Fehler beim Klicken eines Buttons abfangen
    - Fehler beim Öffnen eines Subeditors
      - Fehler beim Öffnen eines Subeditors abfangen
  - Auf Dialoge antworten
    - Antwort für zu erwartende Dialoge setzen
    - Dialoge beim Öffnen eines Subeditors beantworten
  - Weitere Plausibilitätsprüfungen
    - Schreibbare und nicht schreibbare Felder
      - Prüfung auf Feldschreibschutz (änderbar)
    - Leere und nicht leere Felder
      - Prüfung auf leere Felder
    - Werte aus anderen Feldern übernehmen und vergleichen
      - Wert aus anderem Feld übernehmen und vergleichen
      - Wert aus anderem Feld eines anderen Editors übernehmen und vergleichen
      - Wert aus anderem Feld einer Tabellenzeile eines anderen Editors übernehmen und vergleichen
      - Felder editorübergreifend prüfen
  - Referenzieren über Editor-ID
    - Referenzfelder setzen über Editor-ID

    - Optimierung der Testskripte durch Tabellenschreibweise
        - Feldwerte setzen in Tabellenschreibweise
        - anstatt so
        - Setzen und Prüfen von Feldwerten in Tabellenschreibweise
        - anstatt so
        - EditorID in der Tabellenschreibweise
        - Editor^feldName in der Tabellenschreibweise
        - Bearbeiten von Werten in einer Tabelle
    - Scenario Outline
        - Mehrere Artikel anlegen mit Scenario Outline
        - Scenario Outline in Kombination mit Tabellenschreibweise
            - Tabelle, Tabelle, Tabelle!
            - Möglichkeiten mit Platzhaltern: Unterschiedliche Objekte anlegen
        - !dontChange
            - Feldwerte nicht ändern im Zusammenhang mit Scenario Outline
            - Feldwerte nicht ändern im Zusammenhang mit Scenario Outline - Editorübergreifend
    - Infosysteme
        - Test eines Infosystems
        - Test eines Infosystems mit Scenario Outline
        - Test eines Infosystems - Lange Version
        - Test eines Infosystems mit export data und saved values
- Seltener benötigte Steps
    - !lastRow
        - !latRow - Einfaches Beispiel
        - Verwendung von !lastRow bei Feldprüfungen
    - Tippkommandos ausführen
        - Test von Tippkommandos
    - Editoren mit Menüauswahl öffnen
        - Öffnen von Editoren mit Menüauswahl besitzen
    - Editoren wechseln (switch the current editor)
        - Wechseln zwischen Editoren
        - Vor save oder close zu beliebigem Editor wechseln / zweiten Editor parallel öffnen
    - Subeditoren
        - Subeditor öffnen: Absteigen in Auftragsfertigungsliste
        - Sonderfall Subeditor Auftragsfertigungsliste
            - Subeditor öffnen: Absteigen in Auftragsfertigungsliste
    - Editor über Selektionsstring öffnen
        - Öffnen eines Editors über Selektionsstring
    - Tests mit Serverflag
        - Serverflag setzen
    - Fake Date setzen
        - Fake Date setzen
    - FOPs ausführen
        - FOPs ausführen
- Spezielle Steps für Entwickler
    - Test First mit Cucumber
    - Cucumber-Einbindung im Testbett (Testbettdatei, Testausführung, Fehlersuche, EPI-Logs)
    - Referenzausgaben
        - Referenzangabe erzeugen in Feature Datei
        - Freemarker Template Datei EV_VORG_RUECK.ftl
        - Beispiel für eine Ausgabe
        - Testbettdatei ref_ruecklieferung_cu
        - Wann Referenztests einsetzen?
    - Sperrsituationen testen - zweiten Editor parallel öffnen
    - Debuggen Cucumber-Skript → C-Kern
    - Ergebnisse einer Objektauswahl prüfen
        - Ergebnisse einer Objektauswahl
- Tipps zur Skriptgestaltung
    - Skriptaufbau
        - Was ist ein Scenario

    - Kommentare und Beschreibungen
    - Tags
    - Header
  - Best Practice und Vorgaben
- Schlussbemerkungen
  - Vorteile des Tools
  - Abgrenzung
  - Informationsquellen

Einleitung

Einleitung zur Schulung

Udo Stößer:

Herzlich willkommen zur Vorstellung und Schulung des Cucumber Testframeworks.

Zur Motivation: Warum noch ein Testframework?

Immer öfter werden wir gefragt „Woher weiß ich, wo etwas getestet wird“ oder "Warum dauert die Pflege der Tests so lange?". Neben der Komplexität unseres Produkts liegt das vor allem an der Vielzahl der eingesetzten Testmethoden und -werkzeuge. Datenbanklader oder epi sind insbesondere neuen Kolleginnen und Kollegen kaum zu vermitteln.
Der Wunsch nach Vereinfachung und Vereinheitlichung hat einige Kollegen und uns dazu geführt, nach zeitgemäßen Alternativen zu bisherigen Testwerkzeugen zu suchen, diese im Praxiseinsatz zu testen und auf unsere Bedürfnisse anzupassen.

Wir sind überzeugt davon, dass Cucumber das richtige Werkzeug für Anwendungstests ist. Wir wollen die Testlandschaft vereinfachen und daher den Cucumber-Einsatz für die Neuerstellung von Tests in der Anwendungsentwicklung vorgeben.

Anfangsinvest:
Jedes zusätzliche Werkzeug macht zunächst erstmal Einarbeitungsaufwand – das ist uns klar. Wir sind jedoch der Überzeugung, dass durch den Praxiseinsatz in den Teams Cucumber den Lader und großteils auch die direkte Verwendung von edp/epi verdrängen kann und wird.
Schaut Euch die Möglichkeiten von Cucumber an und macht Euch ein Bild von den Features und Anwendungsbeispielen.

Testet es selbst, geht bitte bei inhaltlichen Fragen zu Cucumber auf die Kollegen zu, die sich im letzten dreiviertel Jahr mit der Evaluierung beschäftigt und die Möglichkeiten soweit vorangebracht haben, dass sie euch nun guten Gewissens dieses Testframework an's Herz legen, also Michael, Lisa und Steffen.

Bei grundsätzlichen Fragen rund um den Einsatz von Cucumber stehe in erster Linie ich als Verantwortlicher

für das Qualitätsmanagement aber auch @Steffen Börsig (Deactivated) und @Peter Walser zur Verfügung.

Mail von Jürgen Nöding zum Cucumber-Einsatz

Mail von @Jürgen Nöding vom 04.12.18

Hallo Kollegen,

einige von euch haben Vorbehalte bezüglich des Einsatzes von Cucumber als Testinstrument in der Anwendungsentwicklung geäußert. Sie fürchten unter anderem, dass die Zeit, die sie für Einarbeitung verwenden, den Zeitplan ihrer aktuellen Projekte für 2019r4 gefährdet.

Daher haben wir beschlossen, die Abkündigung von Lader- und epi-Rekorder befristet zurückzunehmen. Der Fokus für neu zu entwickelnde Tests wird dennoch auf Cucumber liegen.

Strategisch werden wir wie folgt vorgehen:

1. Cucumber wird von uns als Test-Tool für alle neuen Integrationstests in der Anwendungsentwicklung ab sofort festgelegt. In Ausnahmefällen ist eine Übergangszeit bis Ende Februar 2019 erlaubt. Bitte bei Bedarf diese Ausnahmefälle mit Udo Stößer oder Pit Walser absprechen.
   Ende Februar werden epi- und Laderrekorder endgültig abgeschafft.
2. Bestehende und anzupassende Tests können derzeit noch im vorhandenen Format weitergeführt werden.
3. Langfristig wollen wir auch Testanpassungen alter Integrationstests nur noch mit Cucumber akzeptieren. Hierzu werden wir zu gegebener Zeit informieren.

--
Mit freundlichen Grüßen / Best regards,
Jürgen Nöding

Handhabung für JUnit-Tests

Auf Nachfrage von Kollegen folgende ergänzende Information zu JUnit-Tests:

Tests im Java und AJO-Umfeld können nach wie vor auch mit JUnit erstellt werden.

[Beschreibung aller Cucumber-Steps]

Funktionsweise von Cucumber

- ***Cucumber*** *ist ein [Behavior-Driven-Development](#)-Werkzeug zur textuellen Spezifikation von Anforderungen an Software und zur automatisierten Überprüfung dieser Beschreibung auf ihre korrekte Implementierung.*
  (Quelle: Wikipedia)

- Umsetzung von Testbefehlen (Steps) für abas ERP
- Basiert auf EPI
- Integriert in Eclipse IDE (graphische Oberfläche)
- Ein Plugin für Entwickler gibt es bereits
- Zur einfachen Installation soll demnächst ein Plugin für Supporter zur Verfügung stehen
- Um die Vorteile des Tools vollumfänglich zu nutzen → Eclipse benutzen!

Basis-Steps Cucumber

Code Completion

Die Tests werden in den Feature-Files geschrieben. Die Tests bestehen aus einzelnen Steps und werden zu einem "Scenario" zusammengefasst. Ein Scenario beschreibt einen Testfall.

Die Syntax folgt dabei immer dem Given-Then-Schema und kann durch AND um weitere Zeilen ergänzt werden.

Erstes einfaches Testskript

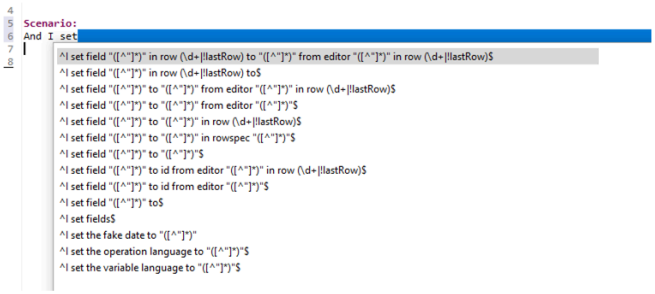
```
Scenario: Summen in Auftrag prüfen
# Vorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "TESTSHOP"
And I create a new row at the end of the table
And I set field "artikel" to "VERKAUF_1" in row 1
And I set field "mge" to "10" in row 1
And I set field "preis" to "5" in row 1
And I create a new row at the end of the table
And I set field "artikel" to "VERKAUF_2" in row 2
And I set field "mge" to "7" in row 2
And I set field "preis" to "2,50" in row 2

# abschließen
And I close the current editor
```

Gebe ein `Given/Then` oder `And` ein, und Du kannst Dir über Strg + Leertaste alle Steps anzeigen lassen, die mit Cucumber möglich sind.

> Leider funktioniert das derzeit im Eclipse nicht (Jira-Issue CUCU-136). Bis dahin kann aber jeder Step auch in der Stepübersicht im Wiki gesucht und gefunden werden.

Tippe weiter (z.B. I set) und die Steps reduzieren sich auf diejenigen, die noch möglich sind:



Eine kurze Beschreibung aller verfügbaren Steps findest Du unter Stepübersicht.

Öffnen eines Editors

Ein Editor kann geöffnet werden. Hierbei können folgende Angaben gemacht werden:

1. Bezeichner des Editors (beliebiger Name)
2. Datenbanktabelle:Datenbankgruppe (AJO Bezeichner)
   `<(Company)>`
   `%,,nummer=;such=;sucherw~/;vdntxts=;vgrtxts=;@gruppe=26;@ablageart=(Active);@autostart=ja <(View)>`
3. Kommando (VIEW, UPDATE, NEW, STORE)
4. Identifizierende Kennung, z.B. Suchname

1. `Given I open an editor "([^"]*)" from table "([^"]*)" with command "([^"]*)" for record "([^"]*)"`
2. `Given I open an editor "([^"]*)" from table "([^"]*)" with command "([^"]*)" for search criteria "([^"]*)"`
3. `Given I open an editor "([^"]*)" from table "([^"]*)" with command "([^"]*)" for record "([^"]*)" and menu choice "([^"]*)"`
4. `Given I open an editor "([^"]*)" for tip command "([^"]*)" and arguments "([^"]*)"`

5. `Given I open an editor "([^"]*)" via ID from editor "([^"]*)" from field "([^"]*)" in row (\d+|!lastRow) for table "([^"]*)" with command "([^"]*)"`
6. `Given I open an editor "([^"]*)" from table "([^"]*)" with command "([^"]*)" for record from editor "([^"]*)"  # Bezug auf einen Datensatz in einem vorangegangenen Editor`

Liste der möglichen Kommandos (Aus der EDP Doku)

Kommando	Bedeutung
NEW	Neuanlage eines Datensatzes
UPDATE	Ändern eines vorhandenen Datensatzes
STORE	Intelligentes Speichern eines Datensatzes, egal ob vorhanden oder nicht (siehe auch unten).
VIEW	Lädt einen Datensatz zum "Zeigen", also ohne Änderungsmöglichkeit
DELETE	Löschen eines Datensatzes
DELIVERY	Datenbankaktion "Lieferung" ausführen
INVOICE	Datenbankaktion "Rechnung" ausführen
REVERSAL	Datenbankaktion "Storno" ausführen
COPY	Neuanlage eines Datensatzes unter Verwendung eines vorhanden Satzes als Vorlage (Kopieren)
REMOVE	Intelligentes Lösche eines Datensatzes, egal ob vorhanden oder nicht
GET	Datenbankaktion "Holen" ausführen
RELEASE	Datenbankaktion "Freigeben" ausführen
PAYMENT	Datenbankaktion "Zahlung" ausführen
CALCULATE	Datenbankaktion "Kalkulieren" ausführen
TRANSFER	Datenbankaktion "Übertragen" ausführen
DONE	Datenbankaktion "Fertig" ausführen
DO	Aufruf eines Tippkommandos (Hierfür gibt es einen separaten Step)

Auf erwartete Feldwerte testen

Berechnete Felder, bzw. Vorbelegungen können auf ihren Inhalt getestet werden:

1. `Then field "([^"]*)" has value "([^"]*)"`
2. `Then field "([^"]*)" has value "([^"]*)" in row (\d+|!lastRow)`

Feldwerte prüfen

```
Scenario: Summen in Auftrag prüfen
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "TESTSHOP"
And I create a new row at the end of the table
And I set field "artikel" to "VERKAUF_1" in row 1
And I set field "mge" to "10" in row 1
And I set field "preis" to "5" in row 1
And I create a new row at the end of the table
And I set field "artikel" to "VERKAUF_2" in row 2
And I set field "mge" to "7" in row 2
And I set field "preis" to "2.50" in row 2

# Prüfen der Positionssumme
Then field "pwert" has value "50.00" in row 1
Then field "pwert" has value "17.50" in row 2

# Prüfung der Nettosumme
Then field "sumnetto" has value "67.50"

# Test abschließen
And I close the current editor
```

Erfolgreiche Testausführung

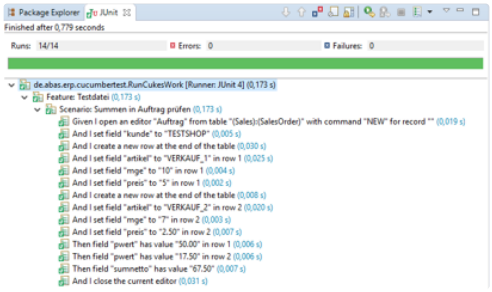
> **Hinweis**
>
> Wie und welche Skripte auf welchen Mandanten ausgeführt werden, kann über RunCukes-Class und die RunConfigurations eingestellt werden. Weitere Informationen gibt der Bereich: [Steuerung der Testausführung über Class RunCukes]

Über grüne Play-Symbol in der Kopfleiste startest Du den Test im jeweiligen Mandanten.

Wird ein Test ausgeführt, öffnet sich am linken Rand ein Fenster. In diesem wird jeder einzelne Step des Tests sowie die Ergebnisse des Durchlaufs anzeigt.

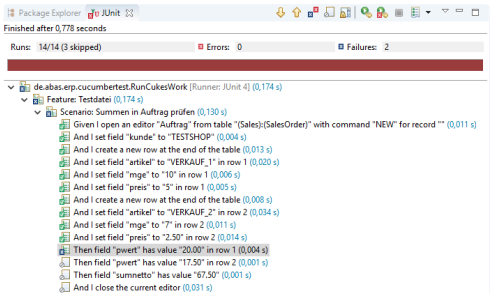
Es gibt drei mögliche Ergebnisse eines Testdurchlaufs:

1. - der Test war erfolgreich

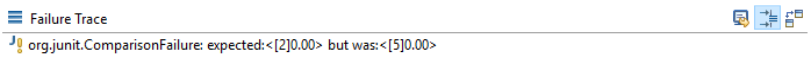


2. Failure - ein zu testender Wert war abweichend

   Z.B.. wurde im Test die Menge 20.00 erwartet, im Testdurchlauf kam aber die Menge 50.00 heraus

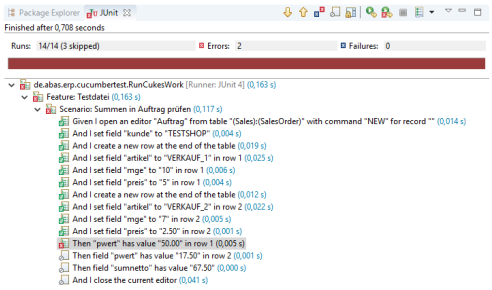


   Im Failure Trace wird die Abweichung beschrieben:



3. Error - ein Fehler (technischer/error) ist aufgetreten

   Z.B. gibt es einen Fehler in einem Step oder eine Aktion ist im Mandanten nicht ausführbar



   Auch in diesem kann der Failure Trace hilfreich sein. In diesem Fall, gibt es den abgegebenen Step nicht (das Wort "field" fehlt).

> **Hinweis**
>
> Wenn einer der beiden Fehlertypen auftritt, werden die restlichen Steps des Scenarios übersprungen. Darum sollten die Tests so gruppiert werden, dass nur direkt abhängige Schritte übersprungen werden.

Auf erwartete Fehler testen

Erwartete Fehler innerhalb eines Test-Scenarios werden über folgende Steps geprüft:

> **Hinweis**
>
> Die ID der Fehlermeldung kann auf zwei Arten herausgefunden werden:
>
> - Suchen des Meldungstextes über den Expertenmodus (*Text Fehlermeldung* ersetzen durch den tatsächlichen Text der Fehlermeldung)
> `!grep "Text Fehlermeldung" $HOMEDIR/` msg.cc `.dic`
> - Beim Durchlauf des Tests wird die Fehlernummer im Failure Trace angezeigt (falls der Test an dieser Stelle scheitert)
> - EDP und Event Exception sind jetzt beide getrennt voneinander abprüfbar, z.B. bei Beleg anfügen, kann auf "Position nicht anfügbar" oder auf "Vorgang abgebrochen" geprüft werden. In der Consolenausgabe erscheinen immer beide Exceptions

Fehler beim Setzen eines Feldwerts

Ein Fehler tritt auf, wenn ein bestimmter Wert ins Feld eingegeben wird. Dies ist sowohl für Felder im Kopf, als auch für Felder in einer Tabelle möglich.

Prüfung auf eine Fehlermeldung:

1. `Then setting field "([^"]*)" to "([^"]*)" throws the exception "([^"]*)"`
2. `Then setting field "([^"]*)" to "([^"]*)" in row (\d+|!lastRow) throws the exception "([^"]*)"`

Welche Fehlermeldung auftritt, kann sowohl über die Eingabe der Fehler-ID sowie über den Text der Fehlermeldung abgefragt werden.

Zudem kann das Setzen von Feldwerten aus anderen Editoren auf eine Fehlermeldung geprüft werden, wenn z.B. der Wert des Feldes nicht bekannt ist. Mehr zum Thema editorenübergreifendes Setzen von Feldwerten wird im Kapitel [Weitere Plausibilitätsprüfungen - Werte aus anderen Feldern übernehmen und vergleichen]() beschrieben.

3. `Then setting field "([^"]*)" in row (\d+|!lastRow) to "([^"]*)" from editor "([^"]*)" in row (\d+|!lastRow)$ throws the exception "([^"]*)"`

Erwartete Fehler bei der Feldeingabe abfangen

```
Scenario: Erwartete Fehlermeldung, im Artikel darf kein Lagerplatz aus externer Lagergruppe als Zugangsplatz
# Testvorbereitung Artikel
Given I open an editor "Artikel" from table "(Part):(Product)" with command "NEW" for record ""
And I set field "such" to "TESTARTIKEL"

# Prüfen Zugangslagerplatz auf eine bestimmte Fehlermeldung
Then setting field "zuplatz" to "L2F1" throws the exception "3912"

# Prüfen Zugangslagerplatz auf eine bestimmte Fehlermeldung
Then setting field "zuplatz" to "L2F1" throws the exception "Der Lagerplatz muss zu der internen Lagergruppe

# Test abschließen
And I close the current editor
```

Fehler beim Speichern des Editors

Tritt der Fehler erst beim Speichern des Editors auf, gilt der folgende Step:

1. `Then saving the current editor throws the exception "([^"]*)"`

Beim Speichern können auch mehrzeilige Texte einer Fehlermeldung geprüft werden:

2. `Then saving the current editor throws the exception`

Der Fehlertext wird in neuer Zeile angegeben:

`Then saving the current editor throws the exception`
`"""`

`XYZ`
`ABC`
`Zeile3`

`"""`

Fehler beim Speichern des Editors abfangen

```
Scenario: Erwartete Fehlermeldung beim Speichern; Identmummer oder Externe Belegnummer in Einkaufsliefersche
# Testvorbereitung Lieferschein Einkauf
Given I open an editor "EK-Lieferschein" from table "(Purchasing):(PackingSlip)" with command "NEW" for reco
And I set field "lief" to "SUPER"
And I create a new row at the end of the table
And I set field "artikel" to "EINKAUF_1" in row 1
And I set field "mge" to "5" in row 1
And I set field "vom" to "."
And I set field "ueb" to "ja"

# Prüfung auf Abfrage Identnummer oder Externe Belegnummer eintragen
Then saving the current editor throws the exception "7054"

# Test abschließen
And I close the current editor
```

Fehler beim Öffnen des Editors mit einem bestimmten Kommando

Der Fehler kann bereits beim Öffnen des Editors kommen, z.B. wenn das Kommando (UPDATE, REVERSAL etc.) nicht möglich ist.

`Then opening an editor from table "([^"]*)" with command "([^"]*)" for record "([^"]*)" throws the exception "([^"]*)"`

`Then opening an editor from table "([^"]*)" with command "([^"]*)" for record from editor "([^"]*)" throws the exception "([^"]*)"`

Durch diesen Step kann man die "Kommandozulässigkeit" prüfen, d.h. zu Testen, dass ein bestimmtes Kommando auf ein Objekt im vorliegenden Zustand nicht ausführbar ist.

Fehler beim Speichern des Editors abfangen

```
Scenario: Erwartete Fehlermeldung beim Stornieren eines Lieferscheins (Beim Oeffnen des Editors)
# Storno des Lieferscheins nicht moeglich, da der Lieferschein noch nicht gebucht ist
Then opening an editor from table "(Sales):(PackingSlip)" with command "REVERSAL" for record "LISTO" throws t
# Exception = Der Lieferschein muss erst gebucht werden.
```

Fehler beim Einfügen oder Löschen von Tabellenzeilen

Test auf Fehler beim Einfügen und Löschen von Zeilen im Tabellenbereich:

1. `Then creating a new row at position (\d+|!lastRow) throws the exception "([^"]*)"`

2. `Then deleting the row at position (\d+|!lastRow) throws the exception "([^"]*)"`

Auch hier können mehrzeilige Fehlermeldungstexte geprüft werden:

3. `Then creating a new row at position (\d+|!lastRow) throws the exception`
4. `Then deleting the row at position (\d+|!lastRow) throws the exception`

Dei Eingabe des Textes erfolgt in der nächsten Zeile:

`Then creating a new row at position 1 throws the exception`
`"""XYZ`
`ABC`
`Zeile3"""`

Abfangen von erwarteten Fehlern beim Erzeugen und Löschen von Tabellenzeilen

```
Scenario: Pruefung Zaehlliste nach Bestandsabschluss (Zeile anfügen, löschen, Einheiten wechseln)
Given I open an editor "Invbearb" from table "(Stocktaking):(EnterQuantitiesCounted)" with command "UPDATE"

And I press button "ladeinvo"

# Exception 3794 - Zeile kann nicht eingefügt werden
Then creating a new row at position 1 throws the exception "3794"

# Exception 3885 - Zeile kann nicht gelöscht werden
Then deleting the row at position 1 throws the exception "3885"

# Test abschließen
And I close the current editor
```

Fehler beim Klick auf Buttons

Ist der Klick auf einen Button in einer Situation nicht erlaubt, wird das durch folgenden Step getestet:

1. `Then pressing button "([^"]*)" throws the exception "([^"]*)"`
2. `Then pressing button "([^"]*)" in row (\d+|!lastRow) throws the exception "([^"]*)"`

Fehler beim Klicken eines Buttons abfangen

```
Scenario: Erwarteter Fehler beim Drücken eines Buttons; Kalkulieren im Auftrag bei leerer Tabelle nicht mögl
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "SUPER"

# Prüfung Button Kalkulieren und Zurückschreiben
    # Fehler 3641: Die Tabelle ist leer
Then pressing button "kalkulr" throws the exception "3641"

# Test abschließen
And I close the current editor

```

Fehler beim Öffnen eines Subeditors

Möchte man über einen BU10-Button einen Subeditor öffnen und erscheint beim Öffnen eines Subeditors ein Fehler, wird der folgende Step dafür eingesetzt:

`Then pressing button "([^"]*)" in row (\d+|!lastRow) to open a subeditor throws the exception "([^"]*)"`

`Then pressing button "([^"]*)" to open a subeditor throws the exception "([^"]*)"`

Fehler beim Öffnen eines Subeditors abfangen

```
Scenario: Erwarteter Fehler beim Öffnen eines Subeditors; das Anlegen einer MZ für Setartikel ist nur für En
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "SUPER"
And I create a new row at the end of the table
And I set field "artikel" to "BG1" in row 1
And I set field "mge" to "5" in row 1

# Prüfung Button MZ für Setartikel
    # Fehler 2634: nur für Teile mit Entnahmeart "über Stückliste" erlaubt
    # Fehler 5705: Kann Submaske nicht öffnen
Then pressing button "mzabsm" in row 1 to open a subeditor throws the exception "2634"
Then pressing button "mzabsm" in row 1 to open a subeditor throws the exception "5705"

# Test abschließen
And I close the current editor

```

Auf Dialoge antworten

Es kommt vor, dass auf Dialoge geantwortet werden muss. Z.B. wird beim Buchen einer Einkaufsrechnung gefragt, ob die Rechnungsabschlusspositionen ergänzt werden sollen.

Hier wird die Dialogantwort und die Dialog-ID vor die Aktion (Step) geschrieben, die den Dialog auslöst:

```
  And I respond with answer "([^"]*)" to the dialog with id "([^"]*)"
```

Die ID des Dialogs wird analog der ID-Suche für Fehlermeldungen durchgeführt (siehe [Cucumber als primäres Testtool#Auf erwartete Fehler testen](#)).

Wird auf einen Dialog geantwortet, so muss dieser auch erscheinen. Kommt der erwartete Dialog nicht so scheitert der Test mit "Dialog answer was given but not used!".

Antwort für zu erwartende Dialoge setzen

```
Scenario: Auf Dialoge antworten; Abfrage der Rechnungsabschlusspositionen in EK-Rechnung
# Testvorbereitung Rechnung Einkauf
Given I open an editor "EK-Rechnung" from table "(Purchasing):(Invoice)" with command "NEW" for record ""
And I set field "lief" to "SUPER"
And I set field "ebeleg" to "Rechnung SUPER"
And I create a new row at the end of the table
And I set field "artikel" to "EINKAUF_1" in row 1
And I set field "mge" to "5" in row 1
And I set field "vom" to "."
And I set field "ueb" to "ja"

# Prüfung auf Abfrage der Rechnungsabschlusspositionen
# Fehlertext 4841 - Rechnungsabschlusspositionen wurden ergänzt - ok?
And I respond with answer "ja" to the dialog with id "4841"
And I save the current editor

# Prüfung der Rechnungsabschlusspositionen
Then field "tename" has value "Nettosumme" in row 2
```

Öffnet sich ein Subeditor in Kombination mit einem Dialog, wir dies in einem Step abgehandelt:

```
And I press button "([^"]*)" to open a subeditor for "([^"]*)" in row (\d+|!lastRow) with dialog "([^"]*)" and answer "([^"]*)"
```

Dialoge beim Öffnen eines Subeditors beantworten

```
Scenario: Beim Öffnen des Subeditors wird die Meldung „Verkehrszahlensatz nicht vorhanden, neu anlegen?" mit
# Kunden öffnen
Given I open an editor "Kunde" from table "(Customer):(Customer)" with command "UPDATE" for record "KUNDE1"

# Planverkehrszahlen im Kunden hinterlegen
And I press button "pvkz" to open a subeditor for "Plan-Vkz" in row 0 with dialog "2011" and answer "ja"

# Speichern und Schhließen
And I save the current subeditor to switch back to the parent editor
And I close the current editor


```

Weitere Plausibilitätstsprüfungen

Schreibbare und nicht schreibbare Felder

Mit folgenden Steps ist im Kopf oder einer Tabellenzeile prüfbar, ob in ein Feld ein Wert eingetragen werden kann oder nicht:

- ```
  Then field "([^"]*)" is modifiable
  ```
- ```
  Then field "([^"]*)" is modifiable in row (\d+|!lastRow)
  ```
- ```
  Then field "([^"]*)" is not modifiable
  ```
- ```
  Then field "([^"]*)" is not modifiable in row (\d+|!lastRow)
  ```

Prüfung auf Feldschreibschutz (änderbar)

```
Scenario: Editierbare Felder prüfen; EK-Rechnung
# Testvorbereitung Rechnung Einkauf
Given I open an editor "EK-Rechnung" from table "(Purchasing):(Invoice)" with command "NEW" for record ""

# Feld lief (Lieferant) editierbar
Then field "lief" is modifiable

# Feld artikel in Zeile editierbar
And I create a new row at the end of the table
Then field "artikel" is modifiable in row 1

# Nachdem ein Lieferant eingetragen wurde ist die Beschaffungsart (bsart) nicht mehr editierbar
And I set field "lief" to "SUPER"
Then field "bsart" is not modifiable

# Das Feld Bezeichnung Artikel (tename) ist nicht editierbar
And I set field "artikel" to "EINKAUF_1" in row 1
Then field "tename" is not modifiable in row 1

# Test abschließen
And I close the current editor
```

In Tabellenschreibweise: Prüfung auf Feldschreibschutz (änderbar)

```
Then field s are modifiable
```

```
Then fields in table are modifiable
```

- Komprimierte Schreibweise in Tabellenform, wenn man mehrere Felder abprüfen möchte auf beschreibbar/schreibgeschützt. Man kann bei der Tabelle auch nur bestimmte Zeilen überprüfen, in dem man in der Spalte !row die Zeilennummer angibt.

```
# Prüfung bei Kopffeldern:

Then fields are modifiable
    | vprg    | ja    |
    | vrab    | nein  |
    | prov    | ja    |
    | eprg    | nein  |
    | erab    | ja    |

# Pfürfung bei Tabellenfelder:

Then fields in table are modifiable
  | !row | vprg  | vrab  | prov  |
  | 1    | ja    | nein  | ja    |
  | 3    | ja    | ja    | ja    |
```

Leere und nicht leere Felder

Wenn nur relevant ist, ob ein Feld einen Wert erhalten hat oder ob es leer ist/geblieben ist (Kopf und/oder Tabellenzeile), dann sind folgende Steps dafür geeignet:

- ```
  Then field "([^"]*)" is empty
  ```
- ```
  Then field "([^"]*)" is empty in row (\d+|!lastRow)
  ```

- ```
  Then field "([^"]*)" is not empty
  ```
- ```
  Then field "([^"]*)" is not empty in row (\d+|!lastRow)
  ```

Prüfung auf leere Felder

```
Scenario: Nicht leere Felder prüfen; Zahlungsbedingungen werden in der EK-Rechnung beim Eintragen eines Lief
# Testvorbereitung Rechnung Einkauf
Given I open an editor "EK-Rechnung" from table "(Purchasing):(Invoice)" with command "NEW" for record ""

# Zahlungsbedingungen zbed wird beim Eintragen des Lieferantes gefüllt
And I set field "lief" to "SUPER"
Then field "zbed" is not empty

# Test abschließen
And I close the current editor
```

Werte aus anderen Feldern übernehmen und vergleichen

Es ist möglich, ein Feld im geöffneten Editor mit einem Wert eines anderen Feldes aus dem selben oder einem anderen Editor zu füllen. Voraussetzung ist, dass der Editor, aus dem der Wert übernommen werden soll, bereits einmal im Feature/Test geöffnet war.

Folgende Möglichkeiten gibt es, um Werte im Kopf aus dem selben oder anderen Editoren zu übernehmen:

1. `And I set field "([^"]*)" to "([^"]*)" from editor "([^"]*)"`
2. `And I set field "([^"]*)" to "([^"]*)" from editor "([^"]*)" in row (\d+|!lastRow)`

   Werte vergleichen:

3. `Then field "([^"]*)" has value equal to field "([^"]*)" from editor "([^"]*)"`
4. `Then field "([^"]*)" has value equal to field "([^"]*)" from editor "([^"]*)" in row (\d+|!lastRow)`

   Der Zusatz *in row* bezieht sich jeweils auf den Editor, in dem das Feld gefüllt werden soll, also in dem gerade geöffneten Editor.

Wert aus anderem Feld übernehmen und vergleichen

```
Scenario: Im gleichen Editor Wert eines Feldes in ein anderes übernehmen und vergleichen; Name des Lieferant
# Testvorbereitung Lieferschein Einkauf
Given I open an editor "EK-Lieferschein" from table "(Purchasing):(PackingSlip)" with command "NEW" for reco
And I set field "lief" to "SUPER"
```

```

# Feldwert in Suchfeld übernehmen
And I set field "such" to "lief^such" from editor "EK-Lieferschein"

# Feldwerte vergleichen
Then field "such" has value equal to field "lief^such" from editor "EK-Lieferschein"

# Test abschließen
And I close the current editor


```

Wert aus anderem Feld eines anderen Editors übernehmen und vergleichen

```
Scenario: Wert eines Feldes aus einem anderen Editor übernehmen; Lagerjournal nach Belegnummer des Liefersch
# Testvorbereitung Lieferschein anlegen
Given I open an editor "Lieferschein" from table "(Purchasing):(PackingSlip)" with command "NEW" for record
And I set field "lief" to "SUPER"
And I set field "ebeleg" to "Lieferschein SUPER"
And I create a new row at the end of the table
And I set field "artikel" to "EINKAUF_1" in row 1
And I set field "mge" to "10" in row 1
And I set field "vom" to "."
And I set field "ueb" to "ja"
And I save the current editor

# Lagerjournal öffnen
Given I open the infosystem "LJ"

# Feld Beleg mit der Identnummer des Lieferscheins füllen
And I set field "beleg" to "nummer" from editor "Lieferschein"

# Test abschließen
And I close the current editor
```

Gibt das Einfügen eines Wertes aus einem anderen Editor einen Fehler zurück, wird das mit folgendem Step gestestet:

```
  Then setting field "([^"]*)" in row (\d+|!lastRow) to "([^"]*)" from editor "([^"]*)" in row (\d+|!lastRow
```

Der folgende Step bietet die Möglichkeit, ein Feld in einer Zeile oder ein Feld im Maskenkopf eines Editors mit einem Feldwert aus einer Zeile oder dem Maskenkopf eines anderen Editors zu füllen. Auch innerhalb eines Editors ist das möglich, wenn jeweils der aktuell geöffnete Editor angebeben wird. Um auf ein Kopffeld zu referenzieren, setze *row 0.*

```
   And I set field "([^"]*)" in row (\d+|!lastRow) to "([^"]*)" from editor "([^"]*)" in row (\d+|!lastRow)
```

Wert aus anderem Feld einer Tabellenzeile eines anderen Editors übernehmen und vergleichen

```
Scenario: Feldwert aus Kopf in Tabellenfeld übernehmen und vergleichen; Im Einkaufslieferschein Verwendung a
# Testvorbereitung Lieferschein Einkauf
Given I open an editor "EK-Lieferschein" from table "(Purchasing):(PackingSlip)" with command "NEW" for reco
And I set field "lief" to "SUPER"
And I create a new row at the end of the table
And I set field "artikel" to "VERKAUF_1" in row 1
And I set field "mge" to "10" in row 1

```

```
# Feldwert in Eigene Bemerkungen (bem) übernehmen
And I set field "bem" in row 0 to "verw" from editor "EK-Lieferschein" in row 1

# Feldwerte vergleichen
Then field "bem" in row 0 has value equal to field "verw" from editor "EK-Lieferschein" in row 1

# Test abschließen
And I close the current editor
```

Um von dem Editor, in dem man sich aktuell befindet, ein Feld in einem anderen Editor des Tests zu prüfen, wird folgender Step verwendet:

`Then field "([^"]*)" from editor "([^"]*)" in row (\d+|!lastRow) has value "([^"]*)"`

Im Beispiel wird aus einem Auftrag ein Lieferschein über eine Teilmenge erstellt. Nach dem Buchen dies Lieferscheins wird im selben Editor "Lieferschein" geprüft, ob sich die offene Liefermenge im im Editor "Auftrag" für diese Position reduziert hat:

Felder editorübergreifend prüfen

```
Scenario: Lieferschein aus Auftrag aktualisiert die ausstehende Liefermenge
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "1"
And I append rows
    | artikel   | mge    | preis   |
    | V1        | 100    | 2,50    |
And I save the current editor
Then field "limge" has value "100"

Given I open an editor "Lieferschein" from table "(Sales):(PackingSlip)" with command "NEW" for record from
And I set fields
    | ueb    | ja    |
    | vom    | .     |
And I set field "mge" to "20" in row 1
And I save the current editor
Then field "limge" from editor "auftrag" in row 1 has value "    80    "
```

Referenzieren über Editor-ID

Für das Setzen der ID eines Objektes stehen eigene Steps zur Verfügung:

1. `And I set field "([^"]*)" to id from editor "([^"]*)"`
2. `And I set field "([^"]*)" to id from editor "([^"]*)" in row (\d+|!lastRow)`

Referenzfelder setzen über Editor-ID

```
Scenario: ID eines Objekts in ein Feld in einem anderen Editor übernehmen; Belegnummer eines Auftrags im Lie
# Kunde anlegen
Given I open an editor "Kunde_MaierAG" from table "(Customer):(Customer)" with command "NEW" for record ""
And I set field "such" to "MaierAG"
And I save the current editor

# Auftrag anlegen
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""

```

```
# Datensatz-ID von Kunde MaierAG einfügen
And I set field "kunde" to id from editor "Kunde_MaierAG"

# Test abschließen
And I close the current editor
```

Optimierung der Testskripte durch Tabellenschreibweise

Wenn in einem Scenario mehrere Feldwerte gesetzt werden, z.B. bei der Anlage von Stammdaten oder als Vorbereitung auf eine Prüfung von Auswirkungen bestimmter Datenkonstellationen, dann wird ist dies durch eine Tabelle einfacher und übersichtlicher. Es muss dann nicht pro Zeile ein ausformulierter Text geschrieben werden, sondern es genügt, den Feldnamen (linke Spalte der Tabelle) und den Feldwert (rechte Spalte der Tabelle) in die Tabelle zu schreiben.
Dazu benötigst Du das Kommando

```
    And I set fields
```

Mit folgendem Kommando werden die Werte nach dem gleichen Prinzip geprüft:

```
    Then fields have values
```

Feldwerte setzen in Tabellenschreibweise

```
Scenario: Feldwerte über Tabellendarstellung setzen; Einkaufsartikel
Given I open an editor "Artikel" from table "(Part):(Product)" with command "NEW" for record ""
And I set fields
    | such      | WENIGZUTIPPEN     |
    | namebspr  | Wenig zu Tippen   |
    | dispoa    | bedarfsbezogen    |
    | chpflicht | ja                |
    | losgr     | 10                |
    | lief      | SUPER             |
    | efrist    | 5                 |
    | epr       | 8.65              |
And I close the current editor
```

anstatt so

```
Scenario: Feldwerte über Tabellendarstellung setzen; Einkaufsartikel
Given I open an editor "Artikel" from table "(Part):(Product)" with command "NEW" for record ""
And I set field "such" to "VIELZUTIPPEN"
And I set field "namebspr" to "Zu viel zu Tippen"
And I set field "dispoa" to "bedarfsbezogen"
And I set field "chpflicht" to "ja"
And I set field "losgr" to "10"
And I set field "lief" to "SUPER"
And I set field "efrist" to "5"
And I set field "epr" to "8.65"
And I close the current editor
```

Es ist möglich mit folgendem Kommando Zeilen in einem Editor anzufügen und zu füllen:

```
    And I append rows
```

Jede Zeile in der Tabelle stellt dann die Zeilen 1, 2, 3 usw. im Editor dar. Die Werte werden in der Tabelle angegeben und in die jeweilige Zelle des Editors übernommen.
In die erste Zeile der Tabelle werden die Feldnamen der im Editor zu füllenden Eingabefelder geschrieben.
Die Zeilen werden automatisch angefügt.
Ein separater Step (And I create a new row at the end of the table) ist nicht nötig.

Um die Werte in einer Tabelle zu prüfen, folgendes Kommando in derselben Logik verwenden:

```
    Then table has values
```

Um in der nachfolgenden Tabelle nur bestimmte Tabellenzeilen zu prüfen, wird die Spalte !row eingefügt. In dieser Spalte kann die Zeile entweder als Nummer oder über Suchkriterien selektiert werden. (siehe Beispiel 2)

Um den Tabelleninhalt komfortabel (Einrückung und richtiges Zahlenformat) aus der GUI zu extrahieren, gibt es das Skript [guitable2cucumber.py]

Setzen und Prüfen von Feldwerten in Tabellenschreibweise

```
Scenario: Tabellen über Tabellendarstellung anfügen; Bestellung
# Testvorbereitung Bestellung
Given I open an editor "Bestellung" from table "(Purchasing):(PurchaseOrder)" with command "NEW" for record
And I set field "lief" to "SUPER"
And I append rows
    | artikel   | mge   | preis |
    | EINKAUF_1 | 10    | 10    |
    | EINKAUF_1 | 20    | 5.50  |
    | EINKAUF_2 | 15    | 4     |
    | EINKAUF_3 | 7     | 2.50  |
And I save the current editor

# Beispiel 1
Then table has values
    | artikel   | pwert |
    | EINKAUF_1 | 100.00|
    | EINKAUF_1 | 110.00|
    | EINKAUF_2 | 60.00 |
    | EINKAUF_3 | 17.50 |

# Beispiel 2
Then table has values
    | !row                      | artikel   | pwert |
    | 1                         | EINKAUF_1 | 100.00|
    | $,,artikel=='EINKAUF-2'   | EINKAUF_1 | 110.00|
    | $,,mge==7                 | EINKAUF-3 | 60.00 |

```

anstatt so

```
Scenario: Tabellen über Tabellendarstellung anfügen; Bestellung
# Testvorbereitung Bestellung
Given I open an editor "EK-Rechnung" from table "(Purchasing):(PurchaseOrder)" with command "NEW" for record
And I set field "lief" to "SUPER"
And I create a new row at the end of the table
```

```
And I set field "artikel" to "EINKAUF_1" in row 1
And I set field "mge" to "10" in row 1
And I set field "preis" to "10" in row 1
And I create a new row at the end of the table
And I set field "artikel" to "EINKAUF_1" in row 2
And I set field "mge" to "20" in row 2
And I set field "preis" to "5.50" in row 2
And I create a new row at the end of the table
And I set field "artikel" to "EINKAUF_2" in row 3
And I set field "mge" to "15" in row 3
And I set field "preis" to "4" in row 3
And I create a new row at the end of the table
And I set field "artikel" to "EINKAUF_3" in row 4
And I set field "mge" to "7" in row 4
And I set field "preis" to "2.50" in row 4
And I save the current editor
Then field "artikel" has value "EINKAUF_1" in row 1
Then field "pwert" has value "100.00" in row 1
Then field "artikel" has value "EINKAUF_1" in row 2
Then field "pwert" has value "110.00" in row 2
Then field "artikel" has value "EINKAUF_2" in row 3
Then field "pwert" has value "60.00" in row 3
Then field "artikel" has value "EINKAUF_3" in row 4
Then field "pwert" has value "17.50" in row 4
```

Auch das Einfügen von Zeilen über append rows sowie die Prüfung von Zeilen sind EditorIDs möglich. Die Schreibweise erfolgt über ein ! gefolgt vom Namen, der dem Editor im Test gegeben wurde:

`!EditorID`

Im folgenden Beispiel gibt es einen Editor, der einen Lagerplatz anlegt, der Editor heißt "MeinPlatz". Zudem wurden zwei Artikel erstellt, die Editoren dazu wurden mit "ArtBG" und "ArtEK1" benannt. Auf diese Editorennamen kann man sich nun beziehen.

EditorID in der Tabellenschreibweise

```
And I append rows
   | artikel | mge  | preis | platz       |
   | V1      | -4   | 6.60  | !MeinPlatz  |
   | V2      | 5    | 3.10  | !dontChange |


Analog auch für then the table has values:

Then table has values
    | art       | zmge  | amge  | detursache                    | storniert |
    | !ArtBG    | -10   |       | Storno-Rückmeldung Fertigung  | nein      |
    | !ArtEK1   |       | -20   | Storno-Rückmeldung Fertigung  | nein      |
```

Es ist zudem möglich, auf Feldwerte dieses angesprochenen Editors zuzugreifen. Dafür wird folgende Schriebweise verwendet:

`!Editor^feldName`

Im Beispiel wird in einen Fertigungsvorschlag die Auftragsnummer aus einem zuvor angelegten Auftrag geschrieben.

Es ist nun auch möglich, über `!Editor^feldName` Feldwerte aus dem angesprochenen Editor mit den Angaben aus dem aktuellen Editor zu vergleichen. Bisher ist das allerdings nur für Kopffelder möglich, Tabellenzeilen können (noch) nicht angesprochen werden.

Editor^feldName in der Tabellenschreibweise

```
Scenario: Die Verwendung aus einem Auftrag in einen Fertigungsvorschlag übernehmen
# Auftrag anlegen
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set fields
    | kunde     | KUNDE1    |
    | vom       | .         |
And I append rows
| artikel       | mge   |
| BG1           | 100   |
And I save the current editor

# Fertigungsvorschlag anlegen und Verwendung aus Auftrag übernehmen
Given I open an editor "fvor" from table "(Purchasing):(WorkOrderSuggestions)" with command "NEW" for record
And I append rows
| artikel   | mge   | verw              |
| BAUGRUPPE | 100   | !Auftrag^nummer   |
And I save the current editor
```

Bestehende Tabellen können mit folgendem Step bearbeitet werden

`And I modify table`

Wichtig dabei ist, dass in der nachfolgenden Tabelle entweder die Tabellenzeile mitgegeben wird (!row als Spalte), entweder als Nummer oder über Suchkriterien.

Über + und - können Tabellenzeilen eingefügt oder gelöscht werden.

Bearbeiten von Werten in einer Tabelle

```
Scenario: Tabellen über Tabellendarstellung bearbeiten; Bestellung
# Testvorbereitung Bestellung
Given I open an editor "Bestellung" from table "(Purchasing):(PurchaseOrder)" with command "NEW" for record
And I set field "lief" to "SUPER"
And I append rows
    | artikel   | mge   | preis |
    | EINKAUF_1 | 10    | 10    |
    | EINKAUF_1 | 20    | 5.50  |
    | EINKAUF_2 | 15    | 4     |
    | EINKAUF_3 | 7     | 2.50  |
And I save the current editor

And I switch the current editor to editor "Bestellung"
# Tabellenwerte ändern
And I modify table
    | !row                              | verw      |
    | 1                                 | 20033_1   |
    | 2                                 | 20033_2   |
    | artikel=='EINLAUF_2' && mge==15   | 20033_3   |
    | -4                                |           |
And I save the current editor
```

Scenario Outline

Wenn ein Scenario mehrfach mit den gleichen Steps aber verschiedenen Parametern durchgeführt werden soll, kann diese Funktion den Code stark verkürzen.

Der Test besteht dann aus dem `Scenario Outline` mit allen Steps sowie den `Examples`. In den Examples werden die Daten gesetzt. **Im Test wird für jede Tabellenzeile der Examples das komplette**

Scenario einmal durchlaufen.

Für Felder, die in jedem Durchlauf auf unterschiedliche Feldwerte gesetzt werden sollen, wird ein Platzhalter (Variable) definiert. Dieser wird in spitze Klammern < > gesetzt. Die Benennung der Platzhalter kann frei gewählt werden.

Im Step und in den Examples müssen die Namen übereinstimmen.

Unterhalb der Platzhalter werden die Feldwerte, für jeden Durchlauf der Examples, gesetzt.

Mehrere Artikel anlegen mit Scenario Outline

```
Scenario Outline: Artikel anlegen
# Artikel anlegen
Given I open an editor "artikel" from table "(Part):(Product)" with command "STORE" for record "<such>"
And I set field "such" to "<such>"
And I set field "namebspr" to "<namebspr>"
And I set field "vpr" to "1000"
And I set field "bsart" to "Eigenfertigung"
And I set field "dispoa" to "<dispoa>"
And I set field "le" to "<le>"
And I set field "chimlager" to "<chimlager>"
And I save the current editor

Examples:

 |row |such   |namebspr                        |dispoa                  |le    |chimlager |
 |01  |ARTVARI|Artikel variantenbezogen        |variantenbezogen        |Stück |Nein      |
 |02  |ARTAUFT|Artikel auftragsbezogen         |auftragsbezogen         |Stück |Nein      |
 |03  |ARTERWB|Artikel erweitert bedarfsbezogen|erweitert bedarfsbezogen|Stück |Nein      |
 |04  |ARTBEDA|Artikel bedarfsbezogen          |bedarfsbezogen          |Stück |Nein      |
 |05  |ARTBESZ|Artikel bedarfsbezogen Satz     |bedarfsbezogen          |Satz  |Ja        |
```

Es ist möglich vor dem “Given...” mit den Examples weitere “Given-Statements” zu setzen. Diese werden aber auch je Example durchlaufen (Meist ist hier ein “STORE” vorteilhafter). Nach dem Examples Abschnitt darf kein Step mehr kommen. Man muss mit einem weiteren Scenario weiter machen.

```
# ---------------------------------------------------------------------------------------------
Scenario Outline: Beispiel
# ---------------------------------------------------------------------------------------------

# Mitarbeiter anlegen
Given I open an editor "Mitarbe" from table "(Employee):(Employee)" with command "STORE" for record "TED"
And I set fields
    | such   | TED |
And I save the current editor

Given I open an editor "<such>" from table "(Part):(Product)" with command "STORE" for record "<such>"
And I set fields
| such     | <such>     |
| namebspr | <namebspr> |
| vkbez    | <vkbez>    |
| vbez     | <vbez>     |
| ebez     | <ebez>     |
| vpr      | <vpr>      |
| bsart    | <bsart>    |
And I save the current editor
Examples: Artikel
| such    | namebspr   | vkbez     | vbez    | ebez      | vpr   | bsart            |
| TECH005 | Schraube 5 | Gewinde 5 | Tech005 | Gewinde 5 | 10000 | Fremdbeschaffung |
| TECH006 | Schraube 6 | Gewinde 6 | Tech006 | Gewinde 6 | 9000  | Fremdbeschaffung |

# Ab hier keine weiteren Statements erlaubt -> Weiter in einem neuen Scenario
```

Scenario Outline in Kombination mit Tabellenschreibweise

Beispiel: Eine Lagerbuchung soll zweimal durchgeführt werden. Einmal für den Artikel EINKAUF_1 und einmal für EINKAUF_2. Im Sceanrio Outline wird darum das Feld "artikel" durch den Platzhalter <artikel> ersetzt. In den Examples unter dem Sceanrio werden die Werte (EINKAUF_1 und EINKAUF_2) für diesen Platzhalter

gesetzt. Weil es sich bei beiden Buchungen um Zugänge handelt, wird "Zugang" fest im Scenario gesetzt. Dies kann auch in den Zeilen zur Lagerbuchung getan werden. Menge und Verwendung werden in jedem Durchlauf mit unterschiedlichen Werten gesetzt und darum über Platzhalter definiert. Der Zugangsplatz ist in beiden Durchläufen gleich.

Tabelle, Tabelle, Tabelle!

```
Scenario Outline: Scenario Outline; Lagerbuchung mit unterschiedlichen Artikeln
Given I open an editor "Lagerbuchung" for tip command "(Stockadjustment)" and arguments ""
And I set fields
    | artikel   | <artikel>  |
    | buart     | Zugang     |
    | beleg     | 999        |
    | beldat    | .          |
And I append rows
    | mge   | platz2  | verw      |
    | <mge> | F2      | <verw>    |
And I save the current editor

Examples:
| artikel   | mge   | verw  |
| EINKAUF_1 | 10    | 1234  |
| EINKAUF_2 | 15    | 1234  |
```

Diese Funktion kann für **jeden** Step eingesetzt werden, ohne ihn anzupassen.

Das Scenario Outline kann auch so eingesetzt werden, dass die Namen der Editoren und Datenbanken ersetzt werden. Voraussetzung ist, dass sich die im Scenario Outline angegebenen Felder in jeder Maske befinden.

Möglichkeiten mit Platzhaltern: Unterschiedliche Objekte anlegen

```
Scenario Outline: Kunden und Lieferanten anlegen
Given I open an editor "<editor>" from table "<table>" with command "STORE" for record "<such>"
And I set fields
    | such | <such> |
    | ans  | <ans>  |
    | str  | <str>  |
    | plz  | <plz>  |
    | nort | <nort> |
And I save the current editor

Examples:
| editor     | table                  | such     | ans                 | str                | plz   | nort
| Kunde      | (Customer):(Customer)  | MAIERAG  | Meier AG            | In den Lissen 33   | 77815 | Bühl
| Kunde      | (Customer):(Customer)  | SCHMIDT  | Schmidt Gmbh&CoKG   | Industriepark 324  | 8000  | München
| Kunde      | (Customer):(Customer)  | KUCKUCK  | Kuckuck AG          | Tannwald 4         | 7933  | Schöngau
| Lieferant  | (Vendor):(Vendor)      | SCHRJUNG | Schrauben Jung      | Gewindestrasse 8   | 4555  | Mutterstа
| Lieferant  | (Vendor):(Vendor)      | GMT      | GMT OHG             | Moritzmatten 45    | 77815 | Bühl
| Lieferant  | (Vendor):(Vendor)      | TUCHART  | Tuch+Art            | Seidenstrasse 4    | 54544 | Leinenhau

```

Auch Steps, die Aktionen in Zeilen ausführen, können genutzt werden. Die Angabe der Zeilennummer wird dann durch *<row>* ersetzt. Alle anderen Platzhalter werden durch die Werte bei Examples ersetzt.

!dontChange

Feldwerte, die bereits gesetzt sind und in einem Scenario nicht geändert werden sollen, werden mit **!dontChange** angegeben. Relevant ist dies vor allem bei Scenario Outlines. !dontChange kann auch beim Setzen editorenübergreifender Werte verwendet werden.

Beispiel:

Feldwerte nicht ändern im Zusammenhang mit Scenario Outline

```
Scenario Outline: !dontChange nutzen, um einen Feldwert nicht zu ändern
Given I open an editor "<such>" from table "(Part):(Product)" with command "STORE" for record "<such>"
And I set fields
    | such     | <such>     |
    | namebspr | <namebspr> |
    | dispoa   | <dispoa>   |
    | lief     | SUPER      |
    | efrist   | 2          |
And I save the current editor

Examples:
| such         | namebspr              | dispoa          |
| AENDERN      | dipsoa ändern         | auftragsbezogen |
| NICHTAENDERN | dispoa nicht ändern   | !dontChange     |
```

Feldwerte nicht ändern im Zusammenhang mit Scenario Outline - Editorübergreifend

```
Scenario Outline: !dontChange nutzen, um einen Feldwert aus einem anderen Editor nicht setzen
# Testvorbereitung Auftrag
Given I open an editor "<editor>" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "TESTSHOP"
And I append rows
    | artikel   | mge | verw   |
    | VERKAUF_1 | 10  | <verw> |
And I save the current editor

Examples:
| editor    | verw         |
| Auftrag1  | Verwendung1  |
| Auftrag2  | Verwendung2  |

Scenario Outline:
# Lagerbuchungen mit und ohne Verwendung
Given I open an editor "Lagerbuchung" for tip command "(Stockadjustment)" and arguments ""
And I set fields
    | artikel   | VERKAUF_1 |
    | buart     | Zugang    |
    | beleg     | 999       |
    | beldat    | .         |
And I create a new row at the end of the table
And I set field "mge" to "<mge>" in row 1

# Editorübergreifend Feldwert setzen
And I set field "verw" in row 1 to "verw" from editor "<editor>" in row 1
And I save the current editor

Examples:
| mge | editor       |
| 10  | Auftrag1     |
| 10  | Auftrag2     |
| 15  | !dontChange  |
```

Infosysteme

Cucumber ist auch sehr gut geeignet, Infosysteme zu testen.

Infosysteme werden mit folgendem Step geöffnet, dabei wird der Name des Infosystems eingesetzt:

```
    I open the infosystem "([^"]*)"
```

Beispiele:

Test eines Infosystems

```
Scenario: Infosysteme öffnen
```

```
# Lagerjournal öffnen
Given I open the infosystem "LJ"
And I close the current editor

```

Test eines Infosystems mit Scenario Outline

```
Scenario Outline: Infosystem EKZENTRALE/VKZENTRALE ablageart=beides
Given I open the infosystem "<IS>"
And I set fields
    | ablageart| beides       |
    | ektyp    | Lieferschein |
    | lsart    | <lsart>      |
    | kebeleg  | <kebeleg>    |
    | vktyp    | <ekvktyp>      |

And I press button "bstart"
Then the table has <sollAnzahlRows> rows
Then field "tlsart" has value "<solltlsart>" in row 1
Then field "ttrans" has value "<solltrans>" in row 1
And I close the current editor

Examples:

|row|IS        |lsart                       |kebeleg    |ekvktyp     |sollAnzahlRows|solltlsart
|01 |EKZENTRALE|!dontChange                 |!dontChange|!dontChange |69            |
|02 |EKZENTRALE|!dontChange                 |151        |!dontChange |1             |Lieferschein
|03 |EKZENTRALE|Rücklieferschein            |!dontChange|!dontChange |33            |Rücklieferschein
|04 |EKZENTRALE|Storno-Rücklieferschein     |!dontChange|!dontChange |6             |Storno-Rücklieferschein
|05 |EKZENTRALE|Stornierter Rücklieferschein|!dontChange|!dontChange |6             |Stornierter-Rückliefers
|06 |VKZENTRALE|!dontChange                 |!dontChange|Lieferschein|63            |
|07 |VKZENTRALE|!dontChange                 |151        |Lieferschein|63            |Rücklieferschein
|08 |VKZENTRALE|Rücklieferschein            |!dontChange|Lieferschein|28            |Rücklieferschein
|09 |VKZENTRALE|Storno-Rücklieferschein     |!dontChange|Lieferschein|6             |Storno-Rücklieferschein
|10 |VKZENTRALE|Stornierter Rücklieferschein|!dontChange|Lieferschein|6             |Stornierter-Rückliefers
```

Test eines Infosystems - Lange Version

```
@Infosystem_EKZENTRALE
Scenario:  Infosystem starten EKZENTRALE, ablageart = beides
Given I open the infosystem "EKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "ektyp" to "Lieferschein"
And I set field "lsart" to ""
And I press button "bstart"
Then the table has 69 rows
And I press button "exp"
Then the table has 96 rows

Scenario:  Infosystem starten EKZENTRALE, ablageart = beides
Given I open the infosystem "EKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "ektyp" to "Lieferschein"
And I set field "lsart" to ""
And I set field "kebeleg" to "151"
And I press button "bstart"
Then the table has 1 rows
Then field "tlsart" has value "Lieferschein" in row 1
Then field "ttrans" has value "E 4" in row 1

Scenario:  Infosystem starten EKZENTRALE, ablageart = beides
Given I open the infosystem "EKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "ektyp" to "Lieferschein"
And I set field "lsart" to "Rücklieferschein"
And I press button "bstart"
Then the table has 33 rows
```

```
Then field "tlsart" has value "Rücklieferschein" in row 1
Then field "ttrans" has value "E 1" in row 1

Scenario:  Infosystem starten EKZENTRALE, ablageart = beides, Storno-rücklieferschein
Given I open the infosystem "EKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "ektyp" to "Lieferschein"
And I set field "lsart" to "Storno-Rücklieferschein"
And I press button "bstart"
Then the table has 6 rows
Then field "tlsart" has value "Storno-Rücklieferschein" in row 1
Then field "ttrans" has value "E +1LS102S1" in row 1

Scenario:  Infosystem starten EKZENTRALE, ablageart = beides, stornierter Rücklieferschein
Given I open the infosystem "EKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "ektyp" to "Lieferschein"
And I set field "lsart" to "Stornierter Rücklieferschein"
And I press button "bstart"
Then the table has 6 rows
Then field "tlsart" has value "Stornierter Rücklieferschein" in row 1
Then field "ttrans" has value "E +1LS102R1" in row 1

@Infosystem_VKZENTRALE
Scenario:  Infosystem starten VKZENTRALE, ablageart = beides
Given I open the infosystem "VKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "vktyp" to "Lieferschein"
And I set field "lsart" to ""
And I press button "bstart"
Then the table has 63 rows
And I press button "exp"
Then the table has 86 rows

Scenario:  Infosystem starten VKZENTRALE, ablageart = beides, Rücklieferschein
Given I open the infosystem "VKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "vktyp" to "Lieferschein"
And I set field "lsart" to "Rücklieferschein"
And I press button "bstart"
Then the table has 28 rows
Then field "tlsart" has value "Rücklieferschein" in row 1
Then field "ttrans" has value "V 300001" in row 1

Scenario:  Infosystem starten VKZENTRALE, ablageart = beides, Storno-Rücklieferschein
Given I open the infosystem "VKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "vktyp" to "Lieferschein"
And I set field "lsart" to "Storno-Rücklieferschein"
And I press button "bstart"
Then the table has 6 rows
Then field "tlsart" has value "Storno-Rücklieferschein" in row 1
Then field "ttrans" has value "V +1LS102S1" in row 1

Scenario:  Infosystem starten VKZENTRALE, ablageart = beides, Stornierter Rücklieferschein
Given I open the infosystem "VKZENTRALE"
And I set field "ablageart" to "beides"
And I set field "vktyp" to "Lieferschein"
And I set field "lsart" to "Stornierter Rücklieferschein"
And I press button "bstart"
Then the table has 6 rows
Then field "tlsart" has value "Stornierter Rücklieferschein" in row 1
Then field "ttrans" has value "V +1LS102R1" in row 1
```

Test eines Infosystems mit export data und saved values

Will man beim Test eines Infosystems verschiedene Einstellungen testen und jedes mal die ausgegebenen Zeilenwerte der Tabelle prüfen, macht es Sinn, die Tabellen in eine Referenzdatei auszugeben.
Hierbei macht es auch Sinn Werte wie Feldlisten und Referenzdatei in einer globalen Variable (Saved Value) zu speichern.

```
@persistent
Feature: Daten fuer CREATEINVOICE anlegen
Background:
# Fuer jedes Szenario die globalen Saved Values festlegen
Given I set saved value "REF_FILE" to "VK.CREATEINVOICE.CU.REF"
Given I set saved value "Feldliste1" to "vlnummer,vkkl2"
Given I set saved value "Feldliste2" to "vlnummer,vkkl2,twarenempf"

...

Scenario:  Infosystem starten - 1
Given I open the infosystem "CREATEINVOICE"
And I press button "bstart"

# Platzhalter fuer REF_FILE wird ersetzt durch VK.CREATEINVOICE.CU.REF
# Ausgabe erfolgt in eine REF Datei
And I append ScenarioHeadline to output file "$REF_FILE"
# Export fields funktioniert aehnlich wie edpexport
#und gibt die Werte ausgerichtet in eine Referenzdatei aus.
And I export fields "$Feldliste1" from table content to output file "$REF_FILE"

Scenario:  Infosystem starten - 2
Given I open the infosystem "CREATEINVOICE"
And I set field "faelligkeit" to "."
And I set field "lsohnezbed" to "ja"
And I press button "bstart"
And I append ScenarioHeadline to output file "$REF_FILE"
And I export fields "$Feldliste2" from table content to output file "$REF_FILE"

Scenario:  Infosystem starten - 3
Given I open the infosystem "CREATEINVOICE"
And I set field "lsohnezbed" to "ja"
And I press button "bstart"
And I append ScenarioHeadline to output file "$REF_FILE"

# Erweitern eines Save Value um weiteren Wert -> Zusaetzliches Feld ausgeben
# Gilt nur in diesem Szenario
And I add to saved value "Feldliste2" value ",treturnus"
And I export fields "$Feldliste2" from table content to output file "$REF_FILE"
...
```

Seltener benötigte Steps

!lastRow

Spielt die Anzahl der Zeilen einer Tabelle keine Rolle für einen Test, aber Werte in der letzten Zeile sollen geändert bzw. geprüft werden, dann kann statt der Angabe der Zeilennummer der Zusatz **!lastRow** angegeben werden.

Beispiel:

!latRow - Einfaches Beispiel

```
@Lieferschein
Scenario: Lieferschein anlegen
Given I open an editor "templief" from table "(Sales):(PackingSlip)" with command "NEW" for record ""
Then I set field "lief" to "1"
Then I set field "nummer" to "1LS009"
Then I set field "such" to "LS009"
Then I set field "vom" to "."
Then I set field "ebeleg" to "111"
And I create a new row at the end of the table
Then I set field "artikel" to "E2" in row !lastRow
Then I set field "mge" to "8" in row !lastRow
And I save the current editor
```

Verwendung von !lastRow bei Feldprüfungen

```
Scenario: !lastRow für die letzte Zeile einer Tabelle nutzen
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "TESTSHOP"
Then field "zbed" has value "201"
And I append rows
    | artikel   | mge   | verw      |
    | VERKAUF_1 | 10    | VERW_1    |
And I save the current editor

# Dispo laufen lassen, um Fertigungsvorschlag zu bekommen
And I open an editor "Dispo" for tip command "(Scheduling)" and arguments ""
And I close the current editor

# Fertigungsvorschläge öffnen, um Fertigungsvorschlag freizugeben
Given I open an editor "fvor" from table "(Purchasing):(WorkOrderSuggestions)" with command "UPDATE" for rec
And I set field "artikel" to "VERKAUF_1"
And I press button "ladetab"

# Prüfen, dass es der FV mit Verwendung lastRow ist
Then field "verw" has value "VERW_1" in row !lastRow

# Den Fertigungsvorschlag freigeben
And I set field "mfreig" to "ja" in row !lastRow

# Test abschließen
And I press button "freig" to open a subeditor for "BA_freigeben"
And I close the current editor
And I switch the current editor to editor "fvor"
And I save the current editor
```

Tippkommandos ausführen

Tippkommandos werden mit folgendem Step geöffnet. Dabei wird der Name des Tippkommandos eingesetzt:

```
    Given I open an editor "([^"]*)" for tip command "([^"]*)" and arguments "([^"]*)"
```

Eine kleine Übersicht der Tippkommandos findest Du hier. Alle Kommandos werden mit dem FOP (s3sys/) KOMLIST aufgelistet.

Beispiel:

Test von Tippkommandos

```
Scenario Outline: Tippkommando ausführen
Given I open an editor "Lagerbuchung" for tip command "(Stockadjustment)" and arguments ""
And I set fields
    | artikel   | <artikel> |
    | buart     | Zugang    |
    | beleg     | 999       |
    | beldat    | .         |
And I append rows
    | mge       | platz2 | verw   |
    | <mge>     | F2     | <verw> |
And I save the current editor

Examples:
| artikel   | buart  | mge | verw  |
| EINKAUF_1 | Zugang | 10  |       |
| EINKAUF_2 | Zugang | 15  | 1234  |
```

Editoren mit Menüauswahl öffnen

Manche Editoren können nur geöffnet werden, wenn zuvor eine Auswahl aus einem Menü getroffen wird. Diese können mit folgendem Step geöffnet werden:

```
    Given I open an editor "([^"]*)" from table "([^"]*)" with command "([^"]*)" for record "([^"]*)" and men
```

Beispiel: Inventureröffnung

Öffnen von Editoren mit Menüauswahl besitzen

```
Scenario: Inventureröffnung
Given I open an editor "Invbearb" from table "(StockTaking):(EnterQuantitiesCounted)" with command "RELEASE"
And I save the current editor
```

Editoren wechseln (switch the current editor)

Innerhalb eines Features kann es vorkommen, dass Änderungen in dem einen Editor vorgenommen werden, anschließend mit einem anderen Editor gearbeitet wird und schließlich wieder Änderungen im ersten Editor anfallen oder etwas geprüft werden soll. Auf diesen ersten Editor kann mit einem And-Step zugegriffen werden. Dabei müssen keine näheren Angaben zum Objekt angegeben werden. Es ist auch möglich, diesen anderen Editor mit einem anderen Kommando zu öffnen (VIEW, UPDATE, etc.).

1. `And I switch the current editor to editor "([^"]*)"`
2. `And I switch the current editor to editor "([^"]*)" with command "([^"]*)"`

Beispiel:

Wechseln zwischen Editoren

```
Scenario: Den Editor wechseln
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "TESTSHOP"
Then field "zbed" has value "201"
And I append rows
    | artikel   | mge   |
    | VERKAUF_1 | 10    |
And I save the current editor

# Zahlungsbedingungen im Kunden ändern
Given I open an editor "Kunde" from table "(Customer):(Customer)" with command "STORE" for record "TESTSHOP"
And I set field "zbed" to "202"
And I save the current editor

# Im Auftrag prüfen, ob die geänderten Zahlungsbedingungen NICHT übernommen wurden
And I switch the current editor to editor "Auftrag"
Then field "zbed" has value "201"
And I close the current editor

# Die Zahlungsbedingungen im Kunden wieder zurücksetzen
And I switch the current editor to editor "Kunde"
And I set field "zbed" to "201"
And I save the current editor
```

Vor save oder close zu beliebigem Editor wechseln / zweiten Editor parallel öffnen

siehe dazu das Kapitel “Sperrsituationen testen...” weitere unten

Subeditoren

Der Klick auf einen Button öffnet häufig einen Subeditor. Dies benötigt eigene Steps:

1. `And I press button "([^"]*)" to open a subeditor for "([^"]*)"`
2. `And I press button "([^"]*)" to open a subeditor for "([^"]*)" in row (\d+|!lastRow)`

Der Name des Subeditors kann, wie auch beim Editor, frei gewählt werden.

Geschlossen wird der Subeditor durch einen spziellen Schließen-Step für Subeditoren, anschließend steht man wieder im Ursprungseditor, aus dem man in den Subeditor gewechselt ist:

`And I save the current subeditor to switch back to the parent editor`

Das Schließen und Speichern eines Subeditord kann auch über den klassischen Schließen- oder Speichern-Step erfolgen. Anschließend wird mit dem folgenden Kommando wieder zum ursprünglichen Editor gewechselt:

```
And I switch the current editor to editor "([^"]*)"
```

Beispiel:

Subeditor öffnen: Absteigen in Auftragsfertigungsliste

```
Scenario: Einen Subeditor öffnen; Fertigungsliste aus einem Auftrag öffnen
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set field "kunde" to "TESTSHOP"
And I append rows
    | artikel   | mge   |
    | VERKAUF_1 | 10    |
    | VERKAUF_2 | 10    |

# Fertigungsliste von VERKAUF_1 aufrufen
And I press button "absteig" to open a subeditor for "Auftragsfertigungsliste" in row 1
# Irgendetwas im Subeditor tun
# Den Subeditor schließen und zum ursprünglichen Editor wechseln
And I close the current editor
And I switch the current editor to editor "Auftrag"

# Den ursprünglichen Editor speichern
And I save the current editor
```

Sonderfall Subeditor Auftragsfertigungsliste

Der Subeditor Auftragsfertigungsliste nimmt eine Sonderstellung ein, da man hier durch eine beliebig tief verschachtelte Stücklistenbeschreibung *navigieren* kann, d.h. es ist möglich in die nächste Baugruppe abzusteigen und dann wieder aufzusteigen. Hierzu werden BU10 Buttons gedrückt und Menüeinträge

ausgewählt. Dabei bleibt man aber immer im gleichen Editor. Nachfolgend ein Skript, dass das Navigieren durch eine Auftragsfertigungsliste darstellt.

Beispiel:

Subeditor öffnen: Absteigen in Auftragsfertigungsliste

```
Scenario: Zwei Mal absteigen aus VK-Auftrag dann wieder aufsteigen.
Given I open an editor "auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I set fields
    | kunde    | 1    |
And I append rows
    | artikel    | mge |
    | V1         | 10  |
# Absteigen auf Stufe 1 der Ferigungsliste
And I press button "absteig" to open a subeditor for "AFL_Stufe_1" in row 1
# Absteigen auf Stufe 2 der Fertigungsliste aus der 3. Zeile
And I press button "absteig" to open a subeditor for "AFL_Stufe_1_pseudo" in row 3
# Zurück mit Speichern, Wechseln zum Editor und Wählen der Aufsteigfunktion
And I save the current editor
And I switch the current editor to editor "AFL_Stufe_1"
And I press button "aufsteig" to open a subeditor for "AFL_Stufe_1_pseudo"
And I save the current editor
And I switch the current editor to editor "AFL_Stufe_1"
And I save the current editor
And I switch the current editor to editor "auftrag"
And I save the current editor
```

Editor über Selektionsstring öffnen

Kann ein Objekt nicht über das Suchwort oder über switch-Editor geöffnet werden, dann kann ein Objekt auch über einen Suchstring geöffnet werden. Dies ist vor allem bei Objekten relevant, die abgelegt sind.

```
    Given I open an editor "([^"]*)" from table "([^"]*)" with command "([^"]*)" for record "([^"]*)" and sea
```

In die search criteria wird der Suchstring angegeben:

Öffnen eines Editors über Selektionsstring

```
Scenario: Editor mit Selektionssring öffnen; Abgelegte Rückmeldung prüfen
# Rückmeldung auf ersten Arbeitsgang
Given I open an editor "Rückmeldung1" from table "(Workorder):(WorkOrders)" with command "DONE" for record "
And I set fields
    | gut       | ja    |
    | sofort    | ja    |
And I save the current editor

# Rückmeldung prüfen
Given I open an editor "Storno1" from table "(Workorder):(WorkOrders)" with command "REVERSAL" for search cr
Then the table has 3 rows
And I close the current editor
```

Achtung: Der Selektionsausdruck muss immer genau einen einzigen Treffer zurückliefen, ansonsten kommt die Fehlermeldung "Suchbegriff nicht eindeutig".
Will man z.B. immer den neusten Datensatz zu einem Suchwort, so kann man den Selektionsstring wie folgt definieren:

```
Given I open an editor "ReStorno" from table "(Sales):(Invoice)" with command "REVERSAL" for search criteria
```

Tests mit Serverflag

Um auf einem Mandanten mit Serverflag zu testen, vor das erste Scenario folgendes einfügen:

Serverflag setzen

```
@persistent
Feature: Tests

Background:
Given I enable the flag 123

Scenario: ...
```

Dadurch werden alle folgenden Scenarios dieses Features mit dem angegeben Serverflag getestet.

Fake Date setzen

Ein Fake Date kann entweder im Background gesetzt werden und ist dann für den gesamten Test gültig oder in einem Scenario, dann ist das Fake Date nur für dieses Scenario gültig. Im nächsten Scenario gilt dann entweder das aktuelle Datum oder ein anderes gesetztes Fake Date. Ein . (Punkt) setzt das Fake Date auf das aktuelle Tagesdatum.

Ist das Fake Date im Background und im Scenario aktiv und lässt im Scenario das Fake Date auf leer, wird der Fake Date Wert aus dem Background gesetzt. Setzt man im Scenario den . (Punkt), wird auch hier das das aktuelle Tagesdatum gesetzt.

!dontChange ist kein zulässiger Wert und führt zu einem Fehler im Test.

Fake Date setzen

```
@persistent
Feature: Tests

Background:
Given I set the fake date to "02.01.95"


Scenario Outline: Buchungsdatum Lieferschein prüfen
Given I set the fake date to "<fakeDate>"
Given I open an editor "Lieferschein" from table "(Purchasing):(PackingSlip)" with command "NEW" for record
And I set field "lief" to "1"
And I set field "vom" to "."
And I create a new row at the end of the table
And I set field "artikel" to "E1" in row !lastRow
And I set field "mge" to "20" in row !lastRow
And I set field "ueb" to "ja"
And I set field "ebeleg" to "123"
When I save the current editor
Then field "vom" has value "<fakeDateSoll>"

Examples:
| fakeDate  | fakeDateSoll  |
| 19.01.18  | 19.01.18      |
| 20.07.18  | 20.07.18      |
|           | 02.01.95      |
| .         | 04.12.18      |  --> heutiges Tagesdatum


```

FOPs ausführen

Es ist möglich in einem Test ein FOP auszuführen. Es muss ein FOP aus dem Lieferumfang sein oder im Falle eines Automatiktests muß das FOP im Ausführungsverzeichnis des Tests vorliegen.

```
And I execute FOP "([^"]*)"$"
```

Beispiel:

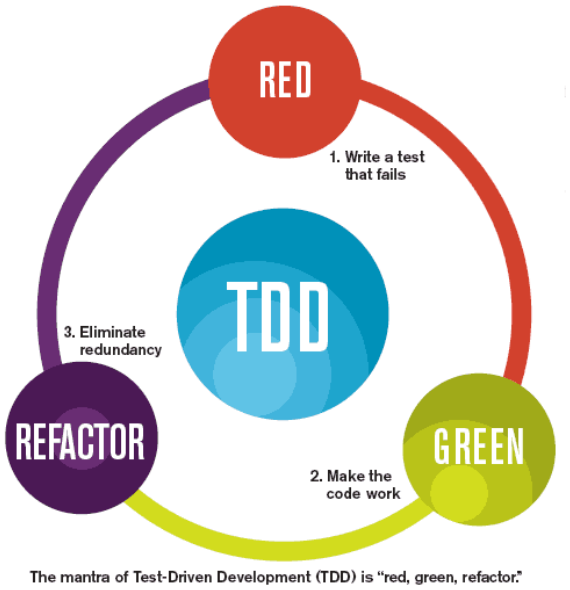
FOPs ausführen

```
Scenario: Ein FOP ausführen
# Testvorbereitung Auftrag
Given I open an editor "Auftrag" from table "(Sales):(SalesOrder)" with command "NEW" for record ""
And I execute FOP "INITAUFTRAG.FOP"
And I set field "kunde" to "TESTSHOP"
Then field "zbed" has value "201"
And I append rows
    | artikel   | mge   |
    | VERKAUF_1 | 10    |
And I save the current editor

```

Spezielle Steps für Entwickler

Test First mit Cucumber



The mantra of Test-Driven Development (TDD) is "red, green, refactor."

- Verwende eigene RunCukes Klasse → RunCukeLocal
- Führe *Micro-Tests* mit neuer Funktionalität gegen den Entwicklungsmandant aus
  Unter Berücksichtigung der Mehrfachausführung → [Tests erstellen für Mehrfachausführung]
- Programmiere den C-Kern
- bis Micro-Test läuft

- Integriere Micro-Test in einen Automatiktest

Cucumber-Einbindung im Testbett (Testbettdatei, Testausführung, Fehlersuche, EPI-Logs)

- Testbett-Datei mit minimalen Befehlen zur Einbindung von Cucumber
  (Anm: Mehrere Feature-Dateien in der Testbettdatei, werden nach alphabetischer Reihenfolge abgearbeitet)
- Feature Datei unter $S/std/test/cucumber/<BEREICH>
- files.txt Einträge

Starten der Tests: mm do_ref_bereich_meintest_cu

Die Fehler stehen in der *err Datei:

- Enthält auch den Aufrufstack von Java
- Wichtigste Fehlerinfos: Fehlertext, Fehlerursache, Fehlerzeile in der Feature-Datei
- EPI-Logs für jedes Scenario

Weitere Infos unter [Cucumber-Tests als Automatiktests]

Referenzausgaben

Wie bei bisherigen Testtools (EDP, EPI, LADER) ist es auch mit Cucumber möglich Referenzangaben zu generieren. Dies ist aber nicht die bevorzugte Testmethode. Das lokale Testen mit Anlegen → Prüfen ist zu bevorzugen. Nur in wenigen Einzelfällen ist der Referenztest vorteilhaft (Mustererkennung). Hierbei sollte aber die Ausgabe begrenzt und human readable bleiben.

Die Ausgabe in eine Datei kann durch Einsatz eines Templates formatiert und dadurch gut leserlich erfolgen.

Referenzangabe erzeugen in Feature Datei

```
...
Scenario: Teilruecklieferung gebuchter Lieferschein
Given I open an editor "teilrueck" from table "(Sales):(PackingSlip)" with command "RETURN" for record "1LS1
Then I set field "nummer" to "1LS100R1"
Then I set field "mge" to "-3" in row 1
Then I set field "fixpwert" to "nein" in row 1
And I save the current editor

# Ausgabe Lieferschein
Given I open an editor "teilrueck" from table "(Sales):(PackingSlip)" with command "VIEW" for record "1LS100

Then I fill template "EV_VORG_RUECK.ftl" and append it to output file "../../VK.EV.RUECKLIEFERUNG.CU.REF"

And I close the current editor
```

Freemarker Template Datei

Freemarker Template Datei EV_VORG_RUECK.ftl

```
<#if dnr == "3">
-- Vorgang Verkauf ------------------------------------------------------------------------------------
</#if>
<#if dnr == "4">
-- Vorgang Einkauf ------------------------------------------------------------------------------------
</#if>
```

```
 7
 8 Typ.........................: ${typa}
 9 Nummer......................: ${nummer}
10 Ablage......................: ${ablagef}
11 <#if typa == "Lieferschein">
12 Lieferscheinart.............: ${lsart}
13 </#if>
14 <#if umplatz != "">
15 Umlagern von................: ${umplatz}
16 </#if>
17 <#if typa == "Rechnung">
18 Rechnung mit Lagerbewegung...: ${fakt}
19 </#if>
20 <#if dnr == "4">
21 Beschaffungsart.............: ${bsart}
22 </#if>
23 <#if (typa == "Lieferschein" || typa == "Rechnung") && (uebertr == "nein")>
24 Gebucht.....................: ja
25 </#if>
26 <#if (typa == "Lieferschein" || typa == "Rechnung")  && (uebertr == "ja")>
27 Gebucht.....................: nein
28 </#if>
29
30 <#if rows??>
31  pos      art       tename               ofmge      mge      pwert       status      ls      re      lifrg
32 ----------------------------------------------------------------------------------------------------------
33 <#list rows as row>
34  ${(row_index+1)?right_pad(9)}${row.artikel?right_pad(10)}${row.tename?right_pad(21)}${row.ofmge?right_pad(1
35 <#if row.abplatz != "">
36  Abgangsplatz:  ${row.abplatz}
37 </#if>
38 </#list>
39 <#else>
40 Keine Positionen!
41 </#if>
```

- Alle Felder des aktuellen Maskenpuffer sind mit ${Feldname} ansprechbar.
- Zeilenfelder werden mit {row.Feldname} angesprochen
- Ausrichtung der Spalten erfolgt mit Methodenaufruf, z.B. ${row.Feldname.right_pad(10)}
- Bedingungen mit <#if Ausdruck> ... </#if>
- Gesamter Template Umfang: [Apache Freemarker Manual]

Beispiel für eine Ausgabe

```
 1 -- Vorgang Verkauf ------------------------------------------------------------------------------------------
 2
 3 Typ.........................: Lieferschein
 4 Nummer......................: 1LS101R4
 5 Ablage......................: nein
 6 Lieferscheinart.............: Rǩcklieferschein
 7 Gebucht.....................: ja
 8
 9  pos      art       tename               ofmge      mge      pwert       status      ls      re      lifrg
10 ----------------------------------------------------------------------------------------------------------
11  1        V12       Verkaufsartikel 12   0          -4       -40000.00   *           0       0       0
12  2        PR.       Rabatte V12          0          0        8000.00     %           0       0       0
13  3        MATZU     Materialzuschlag     0          -4       -0.40       D           0       0       0
14  4        NEPO      Neutrale Position    0          0        0.00                    0       0       0
15  5        NS.       Nettosumme           0          0        -32000.40   #           0       0       0
16  6        ST.       Steuer               0          0        -4800.06    %           0       0       0
```

Vergleich mit einer Referenz direkt in der Featuredatei, mit:

```
Then output file "../../VK.EV.RUECKLIEFERUNG.CU.REF" equals reference file "../../VK.EV.RUECKLIEFERUNG.CU.REF.SOLL"
```

Über die Testbettdatei läßt sich die Referenz wie gewohnt auswerten:

Testbettdatei ref_ruecklieferung_cu

```
NAME ref_ev_ruecklieferung_cu
...

BESCHREIBUNG
...

IN
  #!include cucumber
  DATEI DIRECTMODE
  DATEI DATEI.F
  # Testfaelle Einkauf
  DATEI test/cucumber/ekvkservice/ruecklieferung_vk.feature
  DATEI test/cucumber/templates/EV_VORG_RUECK.ftl
  REF   EV.RUECKLIEFERUNG.CU.REF


INIT
  ${HOMESRC}/snaptool/initCucumberTest.sh
  rm -f *.REF

  # Kleine Verwirrung bei den Verzeichnissen
  mkdir -p ./test/templates
  cp -f ./test/cucumber/templates/*ftl ./test/templates

RUN
  ${HOMESRC}/snaptool/runCucumberTest.sh

  cp ./EK.EV.RUECKLIEFERUNG.CU.REF ./EV.RUECKLIEFERUNG.CU.REF
  cat ./VK.EV.RUECKLIEFERUNG.CU.REF >> ./EV.RUECKLIEFERUNG.CU.REF

  # Möglichkeit die Referenzdatei zu verlängern:
  edpexport.sh -p sy -l 10:1 -k "budat==23.04.1995;@ordnung=id" -f `cat felderljumlagern`  -O "TAB:AUTO"

OUT

DUMP
  taufraeum.sh -g
CMP
CLEAN
```

Dateiablage

`$S/std/test/`

`Referenzdatei Soll: EV.RUECKLIEFERUNG.CU.REF`

cucumber/

templates/

Templatedatei, EV_VORG_RUECK.ftl

`$S/java/projects/cucumber-tests/cucumber/refs/`

`Referenzdateien, wenn man direkt aus Cucumber heraus vergleichen will`

Dateien im Testverzeichnis

`$B/ref_ev_rueckliefer ung_cu/`

/test/cucumber/templates

Templatedatei, EV_VORG_RUECK.ftl

/test/templates

Templatedatei, EV_VORG_RUECK.ftl ???

Wann Referenztests einsetzen?

Referenzausgaben von Tests nur da erzeugen, wo dies wirklich sinnvoll ist. Das Testen "vor Ort" ist vorzuziehen (z.B. bei Plausis). Die Fehlerstelle ist immer auch die verursachende Stelle.

Wann sind Referenzausgaben sinnvoll:

- Wenn nach einer Reihe von Aktionen eine Ergebnismenge betrachtet werden soll, z.B. Folgevorgänge, erzeugte Bewertungen, erzeugte Lagerbewegungen.
  Achtung: Du kannst diese Ausgaben schon während des Tests, an der auslösenden Teststelle (append to output) vornehmen.
- Vermeide die Ausgabe kompletter Feldlisten und aller Datensätze.
  Gib nur die zu testenden Felder und betroffene Datensätze aus. Dann gefährden geringfügige Änderungen (Neuer Vorgang, Neues Feld) die Testausführung nicht.
- Die Ausgabe sollte leicht lesbar sein (tabellarisch oder per Template aufbereitet <> Dump)
- Welche Version ist besser? 😉
- http://cgit.s3.abas.de/cgi-bin/cgit.cgi/erp.git/diff/std/test/STAMMD_LOESCH.REF?id=d139b9b02144bd2a1fbedf3674e35ecc0ac82431
- Ausgabe von Objekten zu verschiedenen Bearbeitungszeitpunkten: Beispiel EK/VK: Es werden Objekte (z.B. Einkaufs-/Verkaufsvorgänge) mit relevanten (sparsam!) Kopf- und Zeilenfeldern ausgegeben. Dadurch kann nach jedem Bearbeitungsschritt (z.B. Teillieferung, Stornierung einer Lieferung, Teilabrechnung, Anzahlungsrechnung, Schlussrechnung....) der Zustand der Objekte ausgegeben und geprüft werden.

Sperrsituationen testen - zweiten Editor parallel öffnen

- Öffne Editor im Modus Ändern → Objekt ist gesperrt
- Mit einem `I'm logged in with password "me"` gibst Du einen weiteren Benutzer vor
- Öffne mit dem zweiten Login einen weiteren Editor und erwarte den Sperrdialog

  ```
  throws a locked object exception
  ```

  (wird abgebrochen)
- Durch hin und her wechseln zwischen den Benutzersitzungen, können so unterschiedliche Sperrsituationen nachgebildet werden.

```
Scenario: Sperrsituation beim Datenbankobjekt Kunde
# Kunde im Modus AENDERN öffnen
Given I open an editor "Kunde1" from table "(Customer):(Customer)" with command "STORE" for record "1"


# Benutzersitzung wechseln
Given I'm logged in with password "me"
# ist jetzt gesperrt -> Sperrdialog erwarten und dann abbrechen
Then opening an editor from table "(Customer):(Customer)" with command "UPDATE" for record "1" throws a lock


# Wieder in erste Benutzersitzung wechseln
Given I'm logged in with password "sy"
# Achtung man muss auf den Editor wechseln!
Given I switch the current editor to editor "Kunde1"
# Editor schliessen -> Kunde freigeben
And I close the current editor

# erneuter zugriff funktioniert
Given I'm logged in with password "me"
And I open an editor "Kunde2" from table "0:1" with command "STORE" for record "1"
And I close the current editor
```

Weiteres Beispiel

- Öffne Editor im Modus Ändern
- Mit einem `I'm logged in  with password  "me"` gibst Du einen weiteren Benutzer vor
- Öffne mit dem zweiten Login einen weiteren Editor und führe so die Sperrsituation herbei
- Am Ende des Scenarios werden alle Editoren geschlossen

```

```

```
# Test fuer Test auf gesperrten Datensatz
Scenario: Ausschreibung mit gesperrten BV
# Sperren eines Bestellvorschlags
Given I open an editor "Bestellvorschläge" from table "(Purchasing):(Item)" with command "UPDATE" for record

# Scenario: Ausschreibung anlegen
Given I'm logged in with password "me"
Given I open an editor "Ausschreibung" from table "(BiddingProcess):(BiddingProcess)" with command "STORE" f
And I set field "such" to "AUSS1"
And I set field "bsart" to "Fremd"
And I create a new row at the end of the table
And I set field "artikel" to "E1" in row 1
And I set field "mge" to "10" in row 1
And I set field "tlief" to "test" in row 1
And I press button "elemaufzu" in row 1
Then field "artikel" has value "E1" in row 1
Then field "lockico" has value "" in row 1
Then field "artikel" has value "E1" in row 2

# Hier steht als Sperr-Icon ein Schloss Symbol drin, wenn der BV gesperrt ist (Das konnte ich bisher nicht t
Then field "lockico" has value "icon:lock" in row 2
And I save the current editor
```

Debuggen Cucumber-Skript → C-Kern

Zum Debuggen verwendest Du die Erweiterungsfähigkeit von Cucumber. Du kannst eigene Steps definieren.

Definiert Du z.B. einen Customized-Step "`Given I enable debugging`",

dann kann dieser an beliebiger Stelle des Skriptes eingebaut werden, um einen Breakpoint im Java Code zu setzen. Bei Ausführung im Debug-Modus des Skripts bleibt der Debugger anschließend am Breakpoint stehen.

Da das Cucumber Skript EPI-Code ausführt, kannst Du wie gewohnt in EPI hinein debuggen und Dich an den Prozess anhängen:

```
# Prozessnummer im Terminal ermitteln:
p mibr2 | grep epi
# gdb an Prozess anhängen
gdb epi 5445
# Breakpoints setzen und in Eclipse fortsetzen
```

Fürs Testbett gibt es auch das Kommando "And I wait for file cucudbg for debugging". Ggf. in der Cucu-

Ergebnisse einer Objektauswahl prüfen

Um Ergebnisse zu prüfen, die unter bestimmten Kriterien in einer Objektwahl angezeigt werden, wird folgender Step verwendet:

```
Given I query "([^"]*)" from table "([^"]*)" where "([^"]*)"
```

Hinter "I query" werden die Felder angegeben, die in der angegebenen Datenbank ausgewertet werden sollen. Hinter "where" werden dann die Selektionskriterien angebenen. Die Selektionskritierien müssen nicht zwingend in der Query mit angegeben werden.

Die Auflistung der Ergebnisse wird mit dem folgenden Step eingeleitet:

```
Then then query has values$
```

Darauf folgt eine Tabelle, die die Felder aus der Query mit den entsprechenden erwarteten Werten beinhaltet.

Ergebnisse einer Objektauswahl

```
Scenario: Query Bestellungen
Given I query "such, budat" from table "(Purchasing):(PurchaseOrder)" where "kl==KETTLER"
Then query has values
| such            | budat           |
| BKETTLER        | 11.03.19        |
| BKETTLER        | 11.03.19        |
```

Tipps zur Skriptgestaltung

Skriptaufbau

Was ist ein Scenario

Ein Scenario sollte optimalerweise einen Testfall beschreiben. Für einen Prozess, in dem z.B. ein Auftrag vorhanden ist, der über einen Lieferschein gebucht wird, anschließend die Rechnung ausgestellt wird, schließlich das Lagerbewegungsjournal für die Vorgänge gelesen, sollte es nicht vier Scenarios geben sondern nur ein Scenario. Das Scenario muss also mehrere Schritte/Aktionen (Given) beinhaltet.

Kommentare und Beschreibungen

Jedes Scenario sollte beschrieben werden. Was genau macht das Scenario. Das Scenario sollte kurz aber aussagekräftig benannt werden. Weitere oder ausführlichere Informationen zum Testfall sollten über Kommentare eingefügt werden. In den Testfällen auf dieser Seite wurden die einzelnen Schritte/Aktionen innerhalb des Scanarios kommentiert, um sie nachvollziehbar zu gestalten. Dies erhöht die Lesbarkeit eines Testskripts und macht es verständlicher für Dritte.

Kommentare werden mit # eingeleitet. Anschließend folgt der Kommentartext. Über Strg + 7 werden alle markierten Zeilen aus-/einkommentiert.

Tags

Tags eignen sich zum Abgrenzen der einzelnen Scenarios. Mit einem @ wird ein Tag initialisiert.

Handelt es sich bei den nachfolgenden Scenarios z.B. um Stammdaten, die für den Testdurchlauf benötigt werden, sollten diese mit `@Testvorbereitung` gekennzeichnet werden.

Die Scenarios sollten mit dem jeweiligen Modul, Projekt, etc. gekennzeichnet werden.

Tags steigern die Nachvollziehbarkeit des Tests. Tags können in den RunCukes aktiviert werden, dadurch laufen nur die Tests, die dieses Tag/diese Tags haben.

```
// Tags mit Komma getrennt, die ausgeführt werden sollen
tags = {"@Testvorbereitung"},
```

Header

Ein Header liefert relevante Informationen zur Feature-Datei:

```
# ****************************************************************************
#  Name             : serabwicklung.feature
#  Autor            : mibr
#  Verantwortlich   : mibr
#  Kontrolle        : as
#  Funktion         : Testet Servicereservierungen, die unabhaengig oder aus einem
#                       Serviceauftrag heraus angelegt werden.
#                       Hierbei wird unterschieden zwischen eintaegigen, mehrtaegigen
#                       und mehrtaegigen ganztag - Terminen.
#
# ****************************************************************************
```

Best Practice und Vorgaben

- **Eclipse IDE verwenden**

- **Tests mehrfachausführbar gestalten**
- **Test First Ansatz mit Cucumber**
- **Feature Dateien einheitlich gestalten (Vorgaben)**

Infos zu diesen Themen auf separaten Seiten:

Cucumber in der Anwendungsentwicklung - Best Practice

Vorgaben für Cucumber-Feature Dateien

Schlussbemerkungen

Vorteile des Tools

- Supporter können manuelle Tests beschleunigen
- Zuarbeit durch Supporter möglich. Tests können mit Supporter besprochen werden.
- Nach der Erstellung können Tests beliebig oft verwendet werden
- Fehlermeldungen lassen sich nach dem Lösen durch einen Entwickler schnell überprüfen
- Lesbar auch für Nicht-Programmierer
- Teile des Tests können ohne Testbett-Overhead ausgeführt werden.
  (Gut für das Ergänzen eines neuen Testfalls - Zeitgewinn)
- Schnelles Auffinden der verursachenden Teststelle im Falle des Scheitern des Tests
- Lokalisierung der Fehler
  Sehr gut geeignet für Oberflächentests und Ablauf von Benutzereingaben (Plausis)
- Test-First-Entwicklung möglich
- Erweiterbar durch Customized-Steps
- Vorteile der Eclipse IDE:
  - Strukturierte Darstellung des gesamten Skripts (Outline)
  - Code Recognition
  - Code Completion (Strg + Leertaste)
  - Übersichtliche Ausgabe des Testergebnisses/der Fehler
  - Editiervorteile von Eclipse (z.B. Code Block Selection Mode, Tastenkombinationen...)

Abgrenzung

- Zu viel Vertrauen auf die Tests könnte zum vernachlässigen des manuellen Testens im System führen
- Gleiche Fehler möglich, wie mit jedem Testtool
  - ID-Falle
  - Ein Test testet zu viel
  - Zu breite Ausgaben
  - Schlechte Vorgänger-Nachfolger-Strategie

- Wie kann verhindert werden, dass zu viel/zu wenig getestet wird und wie können die Tests unabhängiger gestaltet werden (Vorgänger-/Nachfolger-Problem)?
  → Ist hier nicht Thema! Für diese Themen gibt es eine separate Gruppe.

Informationsquellen

- Wiki-Bereich
  - Installationsanleitungen
  - FAQ
  - Stepübersicht
  - Beispielszenarien
  - Vorgaben zur Feature-Gestaltung
  - Cucumber JIRA Board
- Slack-Channel (cucumber)
- Ansprechpartner
  - @Lisa Bettendorf - Support
  - @Siegfried Bücher - Support
  - @Michael Braun -Entwickler
  - @Dominik Pföhler für das Installations-Plugin
- Ansprechpartner für Erweiterungswünsche für Cucumber Steps:
  - @Lisa Bettendorf
  - @Michael Braun